Name bhavya rana

Sapid 590034689

Batch 15

Algorithm

1. Start.
2. Read the size `n` of the array.
3. Declare a variable-length array `nums[n]`.
4. Read `n` elements into the array.
5. Pass the address of `nums` and `n` to `missing_number()`.
6. Inside `missing_number()`:
   - Calculate the expected sum of numbers from `0` to `n` using:
     `expectedSum = n * (n + 1) / 2`
   - Calculate the sum of all elements of the array using a pointer.
   - Find:
     `missing = expectedSum - actualSum`
   - Display the missing number.
7. Stop.

Test Case Table

Test	Size	Array	Expected Output
1	3	`3 0 1`	`Missing number: 2`
2	2	`0 1`	`Missing number: 2`
3	9	`9 6 4 2 3 5 7 0 1`	`Missing number: 8`
4	1	`0`	`Missing number: 1`
5	4	`1 2 3 4`	`Missing number: 0`

C program

```c
#include <stdio.h>

void missing_number(int *nums, int size)
{
    int expectedSum, actualSum = 0, missing;
    int *ptr;

    // Sum of numbers from 0 to size
    expectedSum = size * (size + 1) / 2;

    // Calculate sum of array elements using pointer
    for (ptr = nums; ptr < nums + size; ptr++)
    {
        actualSum = actualSum + *ptr;
    }

    missing = expectedSum - actualSum;

    printf("Missing number: %d\n", missing);
}

int main()
{
    int size, i;

    printf("Size: ");
    scanf("%d", &size);

    int nums[size];

    printf("Array: ");
    for (i = 0; i < size; i++)
    {
        scanf("%d", &nums[i]);
    }

    // Passing array using its address
    missing_number(nums, size);

    return 0;
}
```

Execution

